

2026-05-27 - CBOR Encoding and Sending Data over USB

Goal

Use the CBOR (Concise Binary Object Representation) format to encode structured sensor data on the nRF52840 and transmit it over the USB CDC-ACM virtual serial port established in Day 11.

What you will learn

- What CBOR is and why it is used in embedded systems
- How to enable the zcbor library in Zephyr
- How to encode a map (key-value structure) with zcbor
- How to send the encoded binary frame over USB and decode it on a PC

Overview

CBOR (RFC 8949) is a binary data format similar to JSON but much more compact — ideal for bandwidth-constrained links like USB serial or BLE. Zephyr ships with zcbor, a code-generated CBOR library with a streaming encoder/decoder.

Enable zcbor in prj.conf

```
CONFIG_ZCBOR=y
CONFIG_ZCBOR_STOP_ON_ERROR=y
```

Encoding a simple map

```
#include <zcbor_encode.h>

uint8_t buf[64];
struct zcbor_state_t states[2];
zcbor_new_encode_state(states, ARRAY_SIZE(states), buf, sizeof(buf), 1);

// Encode: {"temp": 2350, "hum": 65}
zcbor_map_start_encode(states, 2);
zcbor_tstr_put_lit(states, "temp");
zcbor_int32_put(states, 2350);           // 23.50 °C * 100
zcbor_tstr_put_lit(states, "hum");
zcbor_int32_put(states, 65);
zcbor_map_end_encode(states, 2);
```

```
size_t len = states[0].payload - buf;
// buf[0..len-1] contains the CBOR frame
```

Sending over USB

```
for (size_t i = 0; i < len; i++) {
    uart_poll_out(usb_uart, buf[i]);
}
uart_poll_out(usb_uart, '\n'); // optional newline as frame delimiter
```

Framing strategy

Raw CBOR is binary — a newline delimiter works for simple cases but is not reliable if the data contains 0x0A. For robust framing, prefix each packet with a 2-byte length:

```
uint8_t header[2] = { (len >> 8) & 0xFF, len & 0xFF };
uart_poll_out(usb_uart, header[0]);
uart_poll_out(usb_uart, header[1]);
for (size_t i = 0; i < len; i++) {
    uart_poll_out(usb_uart, buf[i]);
}
```

Decoding on the PC (Python)

```
import serial, cbor2

port = serial.Serial('/dev/ttyACM0', timeout=1)
raw = port.read(64)          # read up to 64 bytes
data = cbor2.loads(raw)      # {'temp': 2350, 'hum': 65}
print(data)
```

Install: `pip install pyserial cbor2`

zcbor types reference

Type	Encode call
Integer	<code>zcbor_int32_put(state, val)</code>
Text string	<code>zcbor_tstr_put_lit(state, "key")</code>
Byte string	<code>zcbor_bstr_put_arr(state, buf, len)</code>
Boolean	<code>zcbor_bool_put(state, true)</code>
Array start	<code>zcbor_list_start_encode(state, max)</code>
Map start	<code>zcbor_map_start_encode(state, max)</code>

Why this matters

CBOR is the encoding of choice for CoAP, SenML, SUIIT firmware updates, and many IoT protocols. Learning it now prepares you for Bluetooth, LwM2M, and cloud integrations later.

Practice tasks

1. Encode a map with three fields: temp, hum, and a ts (timestamp as uint32).
2. Send it over USB every second and confirm receipt with the Python snippet above.
3. Extend to an array of 5 historical readings inside the same CBOR packet.

Example folder

See `src/2026-05-27_CBOR_over_USB` for the day's code example.

Next topic

Tomorrow we explore the Zephyr Shell subsystem to build an interactive command-line interface over UART or USB.